

Reproducing and Extending Revelio Diff-C Experiments

Based on “Revelio: Interpreting and Leveraging Semantic Information in
Diffusion Models”

Final Project Report

Project Representation Learning

Student: Mherub Ahsan
Matriculation Number: 23457144
Degree Program: M.Sc. Data Science
Institution: Friedrich-Alexander-Universität Erlangen-Nürnberg
Date: 25th September 2026
GitHub Repository: [revelio-prl-reproduction](#)

Abstract

Diffusion models are mainly known for image generation, but recent work has shown that their internal representations also contain useful semantic information. The paper *Revelio: Interpreting and Leveraging Semantic Information in Diffusion Models* investigates this idea by analysing features from different diffusion-model layers and denoising timesteps. In this project, I reproduce and extend selected Diff-C classification experiments from Revelio on the Oxford-IIIT Pet dataset using Stable Diffusion 1.5 as a frozen feature extractor.

The work has two parts. First, I reproduce the layer-wise Diff-C comparison at diffusion timestep $t = 25$. I evaluate four Stable Diffusion 1.5 feature locations: `bottleneck:0`, `up_ft:0`, `up_ft:1`, and `up_ft:2`. The goal of this reproduction is to check whether the same layer-level trend reported in the paper can be observed in my own HPC environment. Second, I perform a small extension based on a gap in the paper’s timestep analysis. The original paper reports results for selected timesteps such as $t = 25$ and $t = 100$, but does not evaluate the intermediate timesteps $t = 50$ and $t = 75$. Therefore, I fix the best reproduced layer, `up_ft:1`, and evaluate $t \in \{25, 50, 75, 100\}$ while keeping the remaining experimental settings fixed.

The reproduction confirms the most important qualitative result of the paper: `up_ft:1` is the strongest evaluated Stable Diffusion 1.5 layer for this fine-grained classification task. It reaches 87.244% test accuracy, close to the paper reference of 88.61%. The own timestep experiment does not confirm the initial hypothesis that intermediate timesteps improve over $t = 25$. Instead, all tested timesteps remain close: $t = 25$ reaches 87.244%, $t = 50$ reaches 86.863%, $t = 75$ reaches 86.890%, and $t = 100$ reaches 86.999%. Thus, within the tested configuration, $t = 25$ remains the best evaluated timestep for `up_ft:1`. These results suggest that early denoising features already contain strong fine-grained semantic information for Oxford-IIIT Pet, while the tested intermediate timesteps do not provide a measurable accuracy improvement.

Contents

1	Introduction	1
2	Background and Related Work	1
3	Method Summary	1
4	Experimental Setup	2
5	Reproduction: Layer Comparison at $t = 25$	2
5.1	Goal	3
5.2	Results	3
6	Own Experiment: Intermediate Timesteps	4
6.1	Motivation and Hypothesis	4
6.2	Experimental Design	4
6.3	Results	4
7	Discussion	6
7.1	Reproduction Deviations	6
7.2	Scope Limitations	6
8	Conclusion	6
A	Execution Evidence	9
B	Selected Log Files	10

1 Introduction

Diffusion models are normally used as generative models, but their internal activations can also serve as visual representations for discriminative tasks. The Revelio paper by Kim, Thomas, and Ghadiyaram studies this question by asking what visual information is represented in different diffusion layers, timesteps, and architectures [1]. The paper combines interpretability through k -sparse autoencoders (k-SAE) with quantitative transfer learning through a lightweight classifier called Diff-C.

This project has two linked goals. The first goal is reproduction: I reproduce the Stable Diffusion 1.5 Diff-C layer comparison on Oxford-IIIT Pet at timestep $t = 25$. This checks whether the important result of the paper can be reproduced on the TinyGPU cluster. The second goal is an own experiment: I investigate a small open question left by the paper’s timestep analysis. The paper reports strong performance for early timesteps, especially $t = 25$, but it does not test the intermediate values $t = 50$ and $t = 75$ in the main reported timestep set. I therefore test whether these intermediate timesteps improve the classification accuracy when the model and layer are fixed.

The report is organized as follows. Section 2 summarizes the paper and the relevant method. Section 4 describes the experimental setup. Section 5 reports the layer-wise reproduction. Section 6 presents the timestep extension. Section 7 discusses the findings and limitations, and Section ?? concludes the report.

2 Background and Related Work

The Revelio paper investigates internal representations of diffusion models. Instead of treating Stable Diffusion only as an image generator, the authors extract intermediate U-Net features and analyze their semantic content. Two complementary tools are used. First, k -sparse autoencoders are trained on diffusion activations to identify interpretable, monosemantic visual features. Second, Diff-C is trained on top of frozen diffusion features to measure how useful those features are for downstream classification.

The key idea is that different layers encode different levels of granularity. In the U-Net structure, the bottleneck has the smallest spatial resolution and tends to capture coarse information. Decoder or upsampling layers, such as `up_ft:1`, preserve more detailed and spatially localized information. For fine-grained tasks such as Oxford-IIIT Pet, this layer is expected to be useful because the label can depend on small visual differences between pet breeds.

The original paper reports that Diff-C trained on Stable Diffusion 1.5 `up_ft:1` features at $t = 25$ reaches 88.61% top-1 accuracy on Oxford-IIIT Pet with empty-prompt conditioning [1]. The paper also shows that classification performance changes with both layer and timestep. For Oxford-IIIT Pet, early timesteps are particularly strong, while later timesteps can lose some fine-grained discriminative information. This motivates a reproduction of the layer effect and a focused experiment on intermediate timesteps.

3 Method Summary

The experimental pipeline follows the Diff-C setup. Images from Oxford-IIIT Pet are passed through a pre-trained Stable Diffusion 1.5 model. Features are extracted from a selected internal U-Net layer at a selected denoising timestep. The extracted feature maps are then used as input to Diff-C, a lightweight convolutional classifier. The diffusion model itself is used as a frozen feature extractor; the classification head is trained for the downstream pet-breed classification task.

Input image $\rightarrow$ **Stable Diffusion 1.5 feature extractor** $\rightarrow$ **selected layer and timestep** $\rightarrow$ **Diff-C classifier** $\rightarrow$ **breed prediction**

Examples: layer $\in \{\text{bottleneck:0}, \text{up_ft:0}, \text{up_ft:1}, \text{up_ft:2}\}$ and timestep $t \in \{25, 50, 75, 100\}$.

Figure 1: High-level pipeline used in this project. The same classifier configuration is used while changing either the layer or the timestep.

The reproduction isolates the layer variable. It fixes $t = 25$ and trains Diff-C on four different Stable Diffusion 1.5 layers. The own experiment isolates the timestep variable. It fixes the model and layer to Stable Diffusion 1.5 `up_ft:1`, then evaluates $t = 25$, $t = 50$, $t = 75$, and $t = 100$.

4 Experimental Setup

All experiments use Oxford-IIIT Pet, a fine-grained image classification dataset of cat and dog breeds [3]. The model is Stable Diffusion 1.5, and features are evaluated using the Diff-C classifier. Unless otherwise stated, the setup follows the paper’s Diff-C training configuration: empty-prompt conditioning, global average pooling, learning rate 1×10^{-4} , and 30 training epochs. Because the project ran on shared TinyGPU hardware, the batch size was set to 4 to fit the available GPUs while keeping it fixed across the final selected experiments.

Table 1: Experimental setup used for the final selected runs.

Item	Setting
Dataset	Oxford-IIIT Pet
Model	Stable Diffusion 1.5
Classifier	Diff-C
Prompt conditioning	Empty prompt
Pooling	Global average pooling (GAP)
Epochs	30
Batch size	4
Learning rate	1×10^{-4}
Metric	Top-1 test accuracy
Cluster	TinyGPU at NHR@FAU
GPU note	RTX 2080 Ti for most runs; V100 for memory-limited layer runs

The experiments were run through SLURM batch jobs on TinyGPU at NHR@FAU. Most runs completed on RTX 2080 Ti nodes. The `bottleneck:0` and `up_ft:0` layer jobs exceeded the memory available on those GPUs and were rerun on V100 GPUs while keeping the final batch size fixed at 4. This is important because it avoids changing the experimental comparison while solving a hardware memory limitation.

The main metric is final top-1 test accuracy at epoch 30. I also record test loss, training accuracy, and training loss from the logs. The final results were extracted from SLURM error logs, because the progress bars and WandB summaries were written to standard error. The generated figures and tables were produced from these extracted log values.

5 Reproduction: Layer Comparison at $t = 25$

5.1 Goal

The reproduction goal is to verify the paper’s claim that `up_ft:1` is the strongest evaluated Stable Diffusion 1.5 layer for fine-grained classification on Oxford-IIIT Pet. I therefore trained Diff-C for four layers at $t = 25$: `bottleneck:0`, `up_ft:0`, `up_ft:1`, and `up_ft:2`. The paper references in this section use the empty-prompt accuracies reported for the same dataset and timestep.

5.2 Results

Table 2 and Figure 2 summarize the layer-wise reproduction. All reproduced values are within the course acceptance band of ± 5 percentage points from the corresponding paper reference values.

Table 2: Layer-wise Diff-C reproduction on Oxford-IIIT Pet using Stable Diffusion 1.5 at $t = 25$.

Layer	Paper (%)	This work (%)	Difference (pp)	Within ± 5 pp?
bottleneck	69.97	71.74	+1.77	Yes
up_ft0	73.29	74.24	+0.95	Yes
up_ft1	88.61	87.24	-1.37	Yes
up_ft2	81.63	79.20	-2.43	Yes

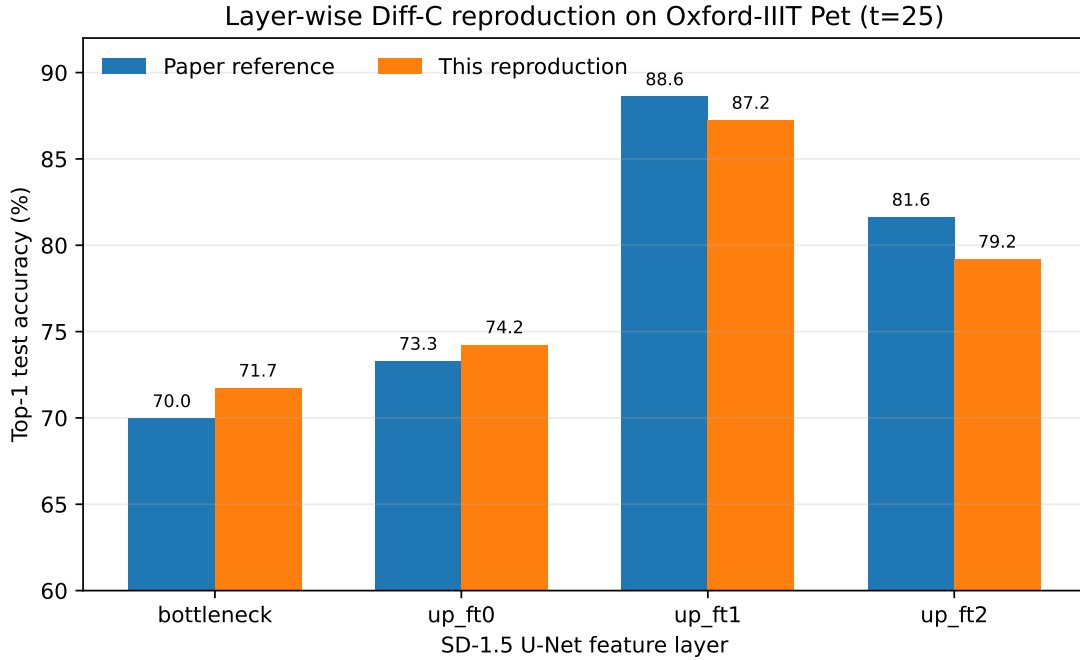


Figure 2: Layer-wise comparison between paper reference values and reproduced values. The most important reproduced trend is that `up_ft:1` is the best evaluated layer.

The reproduced `up_ft:1` result is 87.244%, compared with the paper reference of 88.61%. The difference is -1.37 percentage points, which is small and within the expected reproduction tolerance. The `bottleneck:0` and `up_ft:0` results are also close to the paper references. The `up_ft:2` result is 79.204%, below the paper reference of 81.63%, but still within the accepted deviation.

The qualitative conclusion agrees with the paper: `up_ft:1` is the most useful layer among the evaluated Stable Diffusion 1.5 layers for this fine-grained task. The bottleneck representation is useful but less accurate, and `up_ft:2` does not outperform `up_ft:1`. This supports the interpretation

that useful class-specific semantics are concentrated at an intermediate decoder layer rather than uniformly distributed across the full U-Net.

6 Own Experiment: Intermediate Timesteps

6.1 Motivation and Hypothesis

The paper evaluates a fixed set of timesteps and reports strong performance for early denoising states. However, the intermediate values $t = 50$ and $t = 75$ are not part of the main tested set. This leaves a concrete question: is $t = 25$ genuinely the best local choice for fine-grained classification, or could a slightly noisier intermediate timestep produce features with equal or better semantic content?

My hypothesis was that an intermediate timestep, especially $t = 50$, might match or exceed $t = 25$. The reasoning was that at very early timesteps the image is only slightly noised, so the representation may still contain low-level reconstruction details. A moderately larger timestep could force the model to rely more on abstract semantic information.

6.2 Experimental Design

To test the hypothesis, I fixed the model and layer to the best reproduced setting: Stable Diffusion 1.5 up_ft:1. I then trained Diff-C for four timestep values, $t \in \{25, 50, 75, 100\}$. The new own-experiment values are $t = 50$ and $t = 75$. The values $t = 25$ and $t = 100$ are used as comparison points because they are present in the paper’s reported timestep analysis.

This design changes only one experimental variable, the diffusion timestep. The dataset, model, layer, prompt setting, pooling, training epochs, batch size, learning rate, and metric remain fixed. This makes the timestep comparison controlled and easy to interpret.

6.3 Results

Table 3 reports the final epoch results. Figure 3 shows the final accuracy at each timestep, and Figure 4 shows the validation accuracy curves over training.

Table 3: Timestep sweep for Stable Diffusion 1.5 up_ft:1. The intermediate values $t = 50$ and $t = 75$ are the own experiment.

t	Paper ref. (%)	This work (%)	Test loss	Role
25	88.61	87.24	0.51860	baseline
50	–	86.86	0.55060	own experiment
75	–	86.89	0.53780	own experiment
100	88.29	87.00	0.51220	control

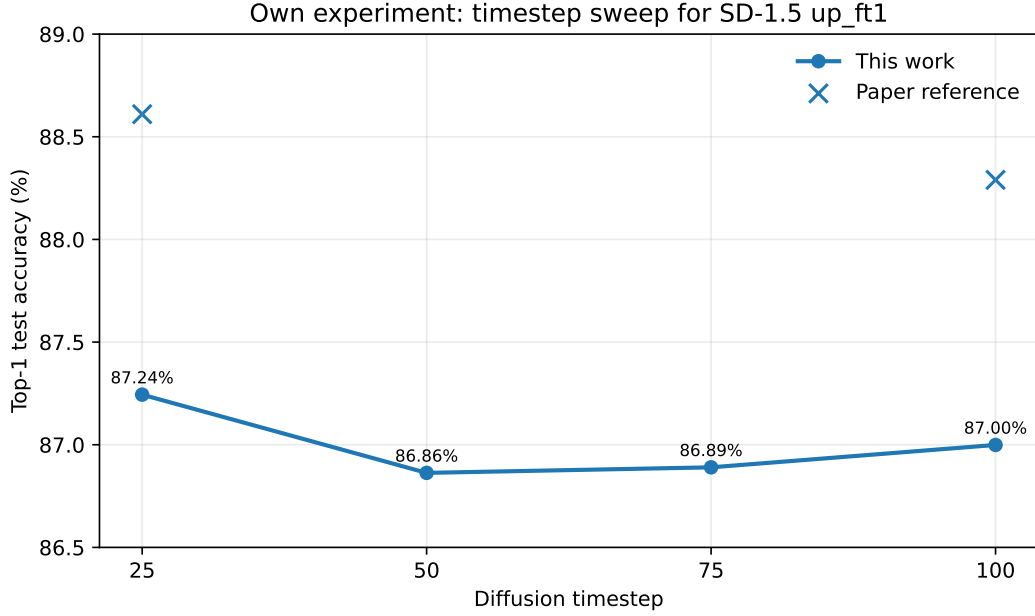


Figure 3: Final top-1 test accuracy for the timestep sweep. The tested intermediate timesteps do not improve over $t = 25$.

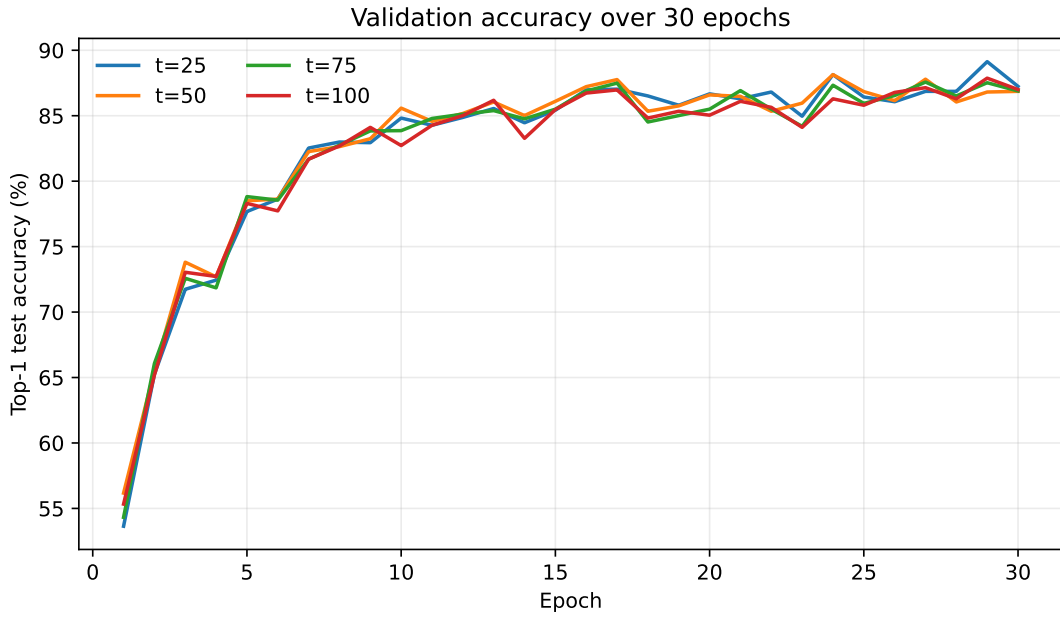


Figure 4: Validation accuracy over 30 epochs for the four timestep runs. The curves converge to a narrow accuracy range.

The best result among the evaluated timesteps is $t = 25$ with 87.244%. The intermediate timestep $t = 50$ reaches 86.863%, and $t = 75$ reaches 86.890%. The $t = 100$ control run reaches 86.999%. Therefore, the hypothesis is not confirmed: neither $t = 50$ nor $t = 75$ improves over $t = 25$.

At the same time, the differences are small. Compared with $t = 25$, the accuracy decreases by 0.381 percentage points at $t = 50$, 0.354 points at $t = 75$, and 0.245 points at $t = 100$. This indicates that `up_ft:1` features are relatively stable over the tested early timestep range. The result does not support a new optimum at $t = 50$ or $t = 75$, but it suggests that the representation remains strong across a small range of denoising steps.

7 Discussion

The project produced two main findings. First, the layer-wise reproduction is successful. It confirms that `up_ft:1` is the strongest evaluated Stable Diffusion 1.5 layer on Oxford-IIIT Pet. This is the central trend expected from the Revelio paper for fine-grained classification. The final reproduced `up_ft:1` value is close to the paper reference and all four reproduced layers stay within ± 5 percentage points of the paper values.

Second, the own experiment answers the gap proposal. The intermediate timesteps $t = 50$ and $t = 75$ do not outperform $t = 25$. This refutes the initial hypothesis that a slightly higher noise level might improve the classification representation. The result instead supports the paper’s observation that early denoising timesteps already provide strong features for fine-grained classification.

The result is also useful because it makes the conclusion more specific. It is not valid to claim that $t = 25$ is globally optimal, because only four timesteps were tested in this project. However, it is valid to say that within the evaluated set $\{25, 50, 75, 100\}$, $t = 25$ is the best reproduced setting for Stable Diffusion 1.5 `up_ft:1` on Oxford-IIIT Pet.

7.1 Reproduction Deviations

Small deviations from the paper are expected. The paper used an NVIDIA RTX A6000 and batch size 16, while this project used TinyGPU resources and batch size 4 because of memory limits. Package versions, GPU type, random initialization, and scheduler behavior can also affect the final values. Importantly, the reproduced layer ranking still captures the main trend, which is more important than exactly matching every number.

The larger practical difficulty was GPU memory. The `bottleneck:0` and `up_ft:0` jobs failed on RTX 2080 Ti nodes with CUDA out-of-memory errors. To keep batch size 4 and avoid changing the experiment, these runs were moved to the V100 partition. This is documented in the execution logs and screenshots included in the source package.

7.2 Scope Limitations

This report focuses on the Diff-C classification part of Revelio. It does not fully reproduce the k-SAE label-purity analysis, all datasets, all timesteps, or all architecture comparisons from the original paper. In particular, SD-2.1 and DiT results are not part of the final selected result set. This narrower scope was chosen to complete a controlled reproduction and a meaningful own experiment under the available compute budget.

A second limitation is that each final result is from a single selected run, not an average over multiple seeds. Therefore, small differences between timesteps should not be over-interpreted. The timestep conclusion is conservative: the tested intermediate timesteps did not show an improvement in this run.

8 Conclusion

This project reproduced and extended selected Diff-C classification experiments from *Revelio: Interpreting and Leveraging Semantic Information in Diffusion Models*. The work focused on using Stable Diffusion 1.5 as a frozen feature extractor for fine-grained classification on the Oxford-IIIT Pet dataset. The project had two parts: first, a layer-wise reproduction at timestep $t = 25$, and second, a small own experiment testing intermediate timesteps that were not evaluated in the main paper results.

In the reproduction part, I compared the layers `bottleneck:0`, `up_ft:0`, `up_ft:1`, and `up_ft:2`. The result confirms the main qualitative trend of the paper: `up_ft:1` is the best evaluated layer for this task. It achieved 87.244% test accuracy, close to the paper reference of 88.61%. The other reproduced layer results were also within the accepted deviation range. This supports the idea that semantic information in Stable Diffusion is not distributed uniformly across all U-Net layers, but is especially useful at the intermediate decoder layer `up_ft:1`.

In the own experiment, I fixed the best reproduced layer, `up_ft:1`, and evaluated $t = 25$, $t = 50$, $t = 75$, and $t = 100$. The motivation was that the paper reports selected timesteps, but does not test the intermediate values $t = 50$ and $t = 75$. My hypothesis was that these intermediate timesteps might give a better balance between image detail and semantic abstraction. The results did not support this hypothesis. The best evaluated setting remained $t = 25$ with 87.244% accuracy, while $t = 50$, $t = 75$, and $t = 100$ reached 86.863%, 86.890%, and 86.999%, respectively.

The main conclusion is therefore that the tested intermediate timesteps do not provide a measurable improvement over $t = 25$ in this setup. However, all four timestep results are close to each other, which suggests that `up_ft:1` features are relatively stable in the early denoising range. This strengthens the observation that early diffusion features already contain strong fine-grained semantic information for Oxford-IIIT Pet.

The project also showed practical reproducibility challenges. Some jobs exceeded the memory of RTX 2080 Ti nodes and had to be rerun on V100 GPUs while keeping the batch size and training settings fixed. In addition, the final metrics had to be extracted carefully from SLURM error logs. These details were documented to make the reproduction process transparent.

The scope of this work was limited to the Diff-C classification part of Revelio. It did not fully reproduce the k-SAE label-purity analysis, all datasets, or all architecture comparisons. Future work could test more timesteps between 0 and 100, repeat the experiments with multiple random seeds, include SD-2.1 and DiT comparisons, and combine the Diff-C results with the k-SAE interpretability analysis. Overall, the project successfully reproduces the key layer-wise trend and shows that the missing intermediate timesteps $t = 50$ and $t = 75$ do not improve over the early timestep $t = 25$ in the tested configuration.

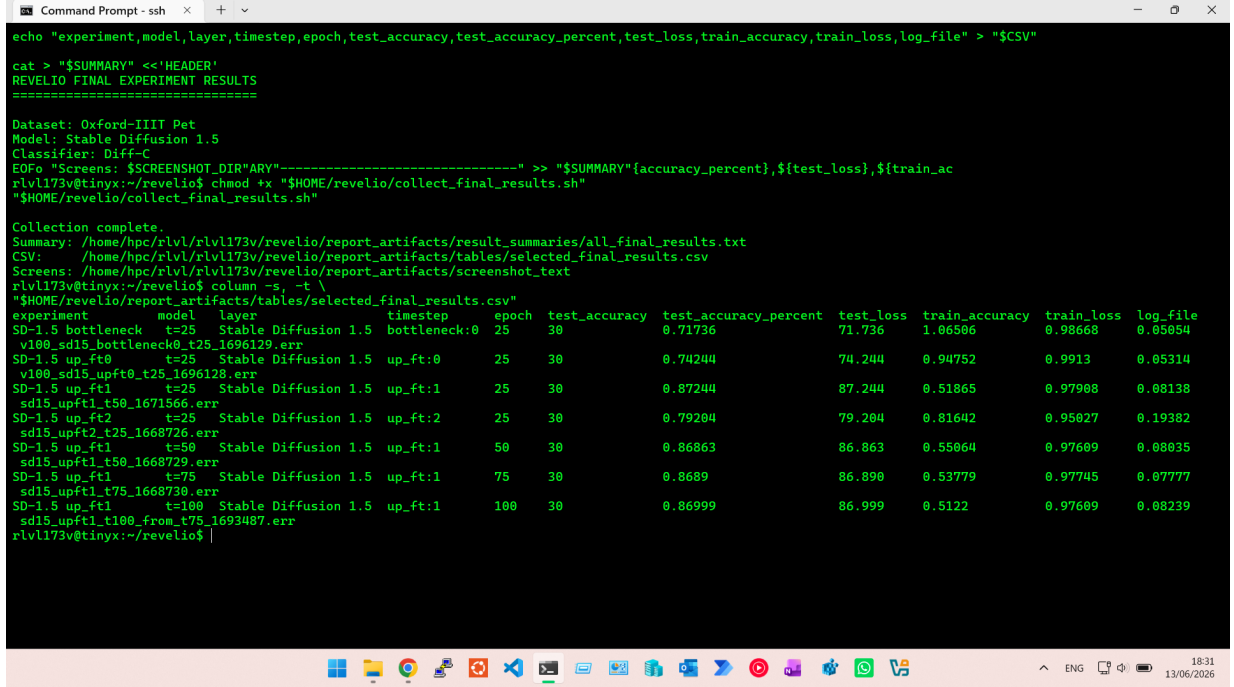
Overall, this project confirms the main reproduced finding that `up_ft:1` is the best evaluated Stable Diffusion 1.5 layer for Oxford-IIIT Pet classification. The own extension shows that the missing intermediate timesteps $t = 50$ and $t = 75$ do not improve over $t = 25$, although they remain close in performance. Thus, within the tested setting, $t = 25$ remains the best evaluated timestep, and early denoising representations appear to be highly effective for fine-grained semantic classification.

References

- [1] D. Kim, X. Thomas, and D. Ghadiyaram. *Revelio: Interpreting and Leveraging Semantic Information in Diffusion Models*. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2025.
- [2] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.
- [3] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar. Cats and dogs. In *CVPR*, 2012.
- [4] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *ICLR*, 2021.
- [5] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

A Execution Evidence

The main report uses extracted numerical logs, CSV tables, and generated figures. The following screenshots are included only as execution evidence and debugging context. They are not required to interpret the main results.



```

echo "experiment,model,layer,timestep,epoch,test_accuracy,test_accuracy_percent,test_loss,train_accuracy,train_loss,log_file" > "$CSV"

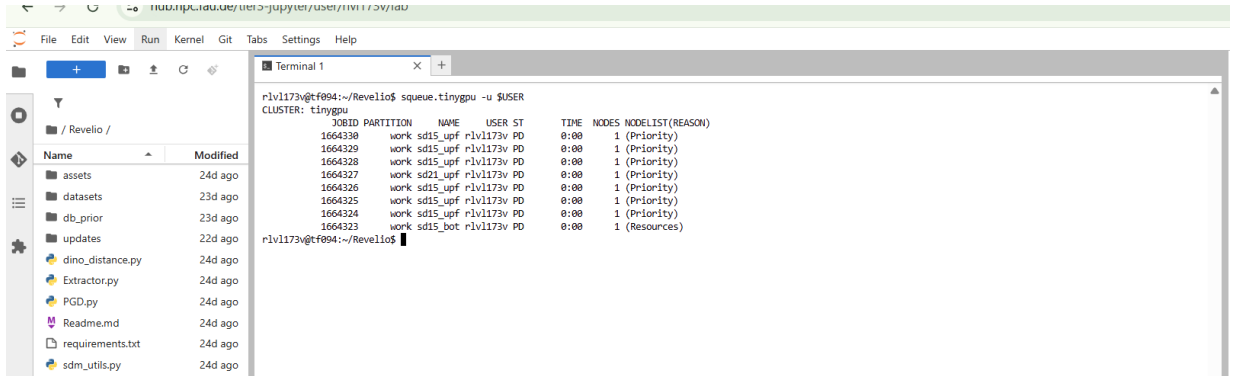
cat > "$SUMMARY" <<'HEADER'
REVELIO FINAL EXPERIMENT RESULTS
=====

Dataset: Oxford-IIIT Pet
Model: Stable Diffusion 1.5
Classifier: Diff-C
EOFo "Screens: $SCREENSHOT_DIR"ARY"-----" >> "$SUMMARY"{accuracy_percent},{test_loss},{train_ac
rlvl173v@tinyx:~/revelio$ chmod +x "$HOME/revelio/collect_final_results.sh"
"$HOME/revelio/collect_final_results.sh"

Collection complete.
Summary: /home/hpc/rlvl/rlvl173v/revelio/report_artifacts/result_summaries/all_final_results.txt
CSV: /home/hpc/rlvl/rlvl173v/revelio/report_artifacts/tables/selected_final_results.csv
Screens: /home/hpc/rlvl/rlvl173v/revelio/report_artifacts/screenshot_text
rlvl173v@tinyx:~/revelio$ column -s, -t \
"$HOME/revelio/report_artifacts/tables/selected_final_results.csv"
experiment      model      layer      timestep    epoch  test_accuracy  test_accuracy_percent  test_loss  train_accuracy  train_loss  log_file
SD-1.5 bottleneck t=25  Stable Diffusion 1.5  bottleneck:0  25  30  0.71736  71.736  1.06586  0.98668  0.85854
v100_sd15_bottleneck0_t25_1696129.err
SD-1.5 up_ft0    t=25  Stable Diffusion 1.5  up_ft:0      25  30  0.74244  74.244  0.94752  0.9913  0.05314
v100_sd15_upft0_t25_1696128.err
SD-1.5 up_ft1    t=25  Stable Diffusion 1.5  up_ft:1      25  30  0.87244  87.244  0.51865  0.97908  0.08138
sd15_upft1_t50_1671566.err
SD-1.5 up_ft2    t=25  Stable Diffusion 1.5  up_ft:2      25  30  0.79204  79.204  0.81642  0.95827  0.19382
sd15_upft2_t25_1668726.err
SD-1.5 up_ft1    t=50  Stable Diffusion 1.5  up_ft:1      50  30  0.86863  86.863  0.55064  0.97609  0.08035
sd15_upft1_t50_1668729.err
SD-1.5 up_ft1    t=75  Stable Diffusion 1.5  up_ft:1      75  30  0.8689   86.890  0.53779  0.97745  0.07777
sd15_upft1_t75_1668730.err
SD-1.5 up_ft1    t=100 Stable Diffusion 1.5  up_ft:1     100  30  0.86999  86.999  0.5122   0.97609  0.08239
sd15_upft1_t100_from_t75_1693487.err
rlvl173v@tinyx:~/revelio$

```

Figure 5: Final extracted result summary displayed in the terminal.



```

rlvl173v@tf094:~/Revelio$ squeue -u $USER
CLUSTER: tinygpu
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
1664330 work sd15_upf rlv173v PD 0:00 1 (Priority)
1664329 work sd15_upf rlv173v PD 0:00 1 (Priority)
1664328 work sd15_upf rlv173v PD 0:00 1 (Priority)
1664327 work sd21_upf rlv173v PD 0:00 1 (Priority)
1664326 work sd15_upf rlv173v PD 0:00 1 (Priority)
1664325 work sd15_upf rlv173v PD 0:00 1 (Priority)
1664324 work sd15_upf rlv173v PD 0:00 1 (Priority)
1664323 work sd15_bot rlv173v PD 0:00 1 (Resources)
rlvl173v@tf094:~/Revelio$

```

Figure 6: Example TinyGPU SLURM queue view from JupyterHub.

```

rlvl173v@tinyx:~$ # Check what failed and why
cat ~/revelio/logs/sd15_bottleneck_t25_1664323.err
cat ~/revelio/logs/sd15_upft0_t25_1664324.err
cat ~/revelio/logs/sd15_upft1_t25_1664325.err
cat: /home/hpc/rlvl/rlvl173v/revelio/logs/sd15_bottleneck_t25_1664323.err: No such file or directory
cat: /home/hpc/rlvl/rlvl173v/revelio/logs/sd15_upft0_t25_1664324.err: No such file or directory
cat: /home/hpc/rlvl/rlvl173v/revelio/logs/sd15_upft1_t25_1664325.err: No such file or directory
rlvl173v@tinyx:~$ grep "Test Accuracy" ~/revelio/logs/*.out | grep "Epoch 90"
rlvl173v@tinyx:~$ # Check errors from all failed jobs
for f in ~/revelio/logs/*.err; do
  echo "=== $(basename $f) ==="
  tail -5 $f
  echo ""
done
=== test_2epoch_1596298.err ===
File "/home/woody/rlvl/rlvl173v/software/private/conda/envs/revelio/lib/python3.10/site-packages/torch/nn/modules/module.py",
line 1527, in _call_impl
  return forward_call(*args, **kwargs)
File "/home/woody/rlvl/rlvl173v/software/private/conda/envs/revelio/lib/python3.10/site-packages/diffusers/models/resnet.py",
line 371, in forward
  output_tensor = (input_tensor + hidden_states) / self.output_scale_factor
torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate 1024.00 MiB. GPU 0 has a total capacity of 10.57 GiB of which
716.12 MiB is free. Including non-PyTorch memory, this process has 9.87 GiB memory in use. Of the allocated memory 8.18 GiB is
allocated by PyTorch, and 1.47 GiB is reserved by PyTorch but unallocated. If reserved but unallocated memory is large try set
ting max_split_size_mb to avoid fragmentation. See documentation for Memory Management and PYTORCH_CUDA_ALLOC_CONF
rlvl173v@tinyx:~$ |

```

Figure 7: CUDA out-of-memory error encountered on RTX 2080 Ti, motivating the V100 reruns for memory-limited layer jobs.

B Selected Log Files

The final selected numerical results were extracted from the following logs:

```

v100_sd15_bottleneck0_t25_1696129.err
v100_sd15_upft0_t25_1696128.err
sd15_upft1_t50_1671566.err      # verified as diffusion_step_25 in log content
sd15_upft2_t25_1668726.err
sd15_upft1_t50_1668729.err
sd15_upft1_t75_1668730.err
sd15_upft1_t100_from_t75_1693487.err

```